Dodatak predavanjima: Korektan tretman znakova bosanskog jezika prilikom ispisa teksta na ekran

Na prvom predavanju je istaknuto da prilikom ispisa teksta na ekranu nećemo koristiti slova izvan engleskog alfabeta, poput slova kao što su "č", "š" itd. koja su specifična za bosanski jezik (i još neke slovenske jezike), jer ispis takvih slova može praviti probleme. Međutim, do problema ne mora uvijek doći. Da li će do problema doći ili ne, ovisi od razvojnog okruženja u kojem je program pisan, zatim od kompajlera koji se koristi za prevođenje programa u mašinski kôd, te od operativnog sistema na kojem se program izvršava. Recimo, posmatrajmo sljedeći program, u kojem se unutar teksta koji želimo ispisati javljaju neka slova izvan engleskog alfabeta, karakteristična za bosanski jezik:

```
#include <iostream>

int main() {
  std::cout << "Ćevabdžinica Željo\ns Baščaršije";
  return 0;
}
```

Ovaj program može, ali i ne mora proizvesti korektan ispis na ekranu. Recimo, ukoliko se ovaj program napiše, kompjalira i pokrene koristeći Linux operativni sistem i razvojna okruženja koja dolaze uz ovaj operativni sistem, vrlo je vjerovatno da će ispis biti koretkan. To će recimo biti slučaj ukoliko se koristi C9/Theia online razvojno okruženje, koje se u pozadini izvršava na nekom serveru pod Linux operativnim sistemom. Tako će, u tom slučaju, ispis na ekranu biti u skladu s očekivanjima, kao što je prikazano na sljedećoj slici:

```
Ćevabdžinica Željo
s Baščaršije
```

S druge strane, ukoliko se koristi neki od MS Windows operativnih sistema, velika je vjerovatnoća da ispis neće biti korektan (mada situacija može zavisiti i od toga koje je razvojno okruženje korišteno za pisanje programa i koji je kompajler korišten za njegovo prevođenje). Recimo, ukoliko se koristi CodeBlocks razvojno okruženje, koje podrazumijevano dolazi zajedno s GCC kompajlerom za C i C++ koji pripada GNU porodici kompajlera, rezultat izvršavanja ovog programa biće kao na sljedećoj slici:

```
─åevabd┼┘inica ┼╝eljo
s Ba┼í─ìar┼íije
```

Ovo može biti uzrok mnogih frustracija, i nerijetko su se mogli čuti komentari poput "položio sam kurs i upisao desetku, a nisam u stanju ni napraviti program koji na ekranu korektno ispisuje moje ime i prezime". Upravo je to bila motivacija za pisanje ovog dodatka. Naime, cilj ovog dodatka je kako postići da ispis teksta koji sadrži specifična slova bosanskog jezika bude korektan neovisno od operativnog sistema na kojem se program izvršava, te neovisno od razvojnog okruženja koje je korišteno za pisanje programa i kompajlera. Pri tome, ograničićemo se samo na problem ispisa, a ne proizvoljne obrade teksta koji sadrži slova izvan engleskog alfabeta, što je znatno složenija tematika.

Da bismo vidjeli kako riješiti ovaj problem, moramo prvo shvatiti zbog čega ovaj problem uopće nastaje. Srž problema je u tome što u doba kada su jezici C i C++ nastali, niko nije vodio mnogo brige o slovima izvan engleskog alfabeta, s obzirom da je tada glavnina računara korištena u zemljama koje pripadaju engleskom govornom području. Poznato je da se svi znakovi u računarskoj memoriji čuvaju u vidu brojčanih šifri (kodova), pri čemu je već dugi niz godina gotovo univerzalno prihvaćeno da se za

slova engleskog alfabeta koriste tzv. ASCII šifre (mada standardi jezika C i C++ to ne naređuju striktno, odnosno teoretski su dopuštene i drugačije šifre, ali to se ne susreće već dugi niz godina). ASCII sistem šifriranja propisuje da se svakom znaku dodjeljuje osmobitna (jednobajtna) šifra. Recimo, malom slovu "a" dodjeljuje se šifra 01100001 (binarno zapisano), odnosno 97 prevedeno u dekadni brojni sistem. Nažalost, ASCII šifre ne predviđaju nikakva slova izvan engleskog alfabeta. Kada su se računari počeli masovno koristiti izvan zemalja engleskog govornog područja, pojavio se problem reprezentacije znakova izvan engleskog alfabeta. Dugo vremena, za rješavanje tog problema koristila su se različita, međusobno nesaglasna rješenja, prilagođavana pojedinim, ali ne svim jezicima. Najviše se koristio sistem šifriranja nazvan ISO/IEC 8859, koji predstavlja proširenje ASCII sistema šifriranja, pri čemu šifre i dalje ostaju osmobitne. Ovaj sistem ima brojne podvarijante, prilagođene raznim jezicima, ali ne svim, što je razumljivo, jer se jasno da se s 8 bita može kodirati najviše $2^8 = 256$ različitih znakova. Tako je podvarijanta ISO/IEC 8859-1 (poznata i kao Latin 1) uključila podršku za jezike kao što su francuski, španjolski, švedski, itd. ali ne i za bosanski jezik. S druge strane, podvarijanta ISO/IEC 8859-2 (poznata i kao Latin 2) uključuje podršku za latinične znakove svih slovenskih jezika (bosanski, češki, poljski, itd.) kao nekih neslovenskih jezika, kao što su recimo njemački, finski i albanski. Recimo, prema ovom sistemu šifriranja, velikom slovu "Č" dodijeljena je šifra 11011000 (binarno zapisano), odnosno 216 prevedeno u dekadni brojni sistem. Postoje i podvarijante koje uključuju slova iz nekih nelatiničnih pisama, kao što je ćirilično, grčko, arapsko, hebrejsko, itd. Međutim, svaka od tih podvarijanti podržava samo do 256 različitih znakova, tako da nastaje problem ukoliko treba istovremeno uključiti recimo slova iz bosanskog i francuskog jezika. Također, mnogi jezici i pisma (poput kineskog jezika i pisma) nisu podržani ovim sistemom. Konačno rješenje nađeno je u sistemu nazvanom UNICODE, koji predviđa vrlo bogat skup znakova (više od 155000 različitih znakova) koji uključuje sve poznate jezike i sva poznata pisma, kao i brojne specijalne znakove (recimo, razne matematičke simbole, emotikone, itd.). Svaki znak predviđen UNICODE sistemom ima svoju jedinstvenu kodnu oznaku (šifru), koja se naziva *kodna tačka* (engl. *code point*). Te šifre nisu nužno jednake šiframa koje se koriste za zapis znaka u računarskoj memoriji. Postoji više različitih varijanti kako se u računarskoj memoriji zaista zapisuju UNICODE znakovi. U većini svjetskih zemalja, danas je najpopularnija varijanta poznata pod nazivom UTF-8. Kod ovog sistema šifriranja, različitim znakovima odgovaraju šifre s različitim brojem bita (8, 16, 24 ili 32, zavisno od znaka). Međutim, ovaj sistem je posve kompatibilan s ASCII sistemom šifriranja, u smislu da svaki znak koji ima svoju ASCII šifru ima osmobitnu UTF-8 šifru koja je potpuno identična njegovoj ASCII šifri (koja je, pored toga, i identična s kodnom tačkom tog znaka). Za znakove koji nemaju ASCII šifru, općenito se kodna tačka i šifra u memoriji razlikuju. Recimo, kodna tačka velikog slova "Č" je 268, dok je, prema UTF-8 varijanti šifriranja, ovaj znak u računarskoj memoriji zapisan kao šesnaestobitni broj 1101010010001101, odnosno 54413 u dekadnom brojnom sistemu. Razlog zbog kojeg UTF-8 nije popularan recimo u Kini je činjenica da UTF-8 sistem svim znakovima kineskog pisma propisuje 24-bitne šifre, što povećava zauzeće memorije u odnosu na recimo UTF-16 sistem, koji znakovima kineskog pisma pripisuje 16-bitne šifre (s druge strane, UTF-16 propisuje 16-bitne šifre i znakovima koji imaju ASCII šifre, tako da sistem UTF-16 nije kompatibilan s ASCII sistemom).

Nakon ovog uvodnog objašnjenja vezanog za način kodiranja znakova u računarskoj memoriji, posebno onih izvan engleskog alfabeta, možemo objasniti i razlog zbog kojeg nastaju problemi prilikom ispisa karakterističnih slova bosanskog jezika. Kada bi se svugdje konzistentno koristio jedan sistem kodiranja, recimo UTF-8, problema ne bi bilo. Međutim, to nije slučaj. Jedan razlog je nepostojanje svjetskog konsenzusa o sistemu kodiranja, a drugi je kompatibilnost s ranije korištenim sistemima kodiranja. Da li će dolaziti do problema prilikom pokušaja ispisa slova izvan engleskog alfabeta (i, općenito, znakova koji nisu podržani ASCII sistemom šifriranja) ili ne, pretežno zavisi od odgovora na sljedeća četiri pitanja:

1. Koji sistem šifriranja neengleskih slova koristi editor koji se koristi za pisanje programa (recimo, editor koji dolazi u sastavu razvojnog okruženja u kojem se piše program)?
2. Koji sistem šifriranja neemgleskih slova podrazumijeva kompajler kada čita tekstualni sadržaj izvornog programa (tj. onog napisanog pomoću editora) koji treba prevesti u mašinski kôd?
3. Koji sistem šifriranja neengleskih slova će koristiti kompajler u generiranom prevedenom mašinskom kôdu (izvršnom programu) koji odgovara izvornom programu?
4. Koji sistem šifriranja neengleskih slova podrazumijeva konzola (tj. dio operativnog sistema koji je, između ostalog, zadužen za ispis teksta na ekran) kada se, tokom izvršavanja programa, bude vršio ispis teksta na ekran?

Ukoliko su odgovori na sva ova četiri pitanja identični, problema neće biti. Međutim, problemi će sigurno nastati ukoliko je odgovor na makar jedno od ova četiri pitanja različit u odnosu na odgovore

na ostala pitanja. Recimo, uz korištenje C9/Theia online razvojnog okruženja pod Linux operativnim sistemom, odgovor na sva četiri pitanja je UTF-8, tako da problema nema. Zapravo, odgovor na prvo pitanje je kod većine modernih editora (mada ne svih) upravo UTF-8. Recimo, Notepad editor koji dolazi uz novije verzije MS Windows operativnih sistema podrazumijevano neengleska slova čuva koristeći UTF-8 šifriranje, ukoliko se u opcijama koje se zadaju prilikom snimanja ne kaže drugačije. Editor koji dolazi uz CodeBlocks razvojno okruženje na MS Windows operativnim sistemima podrazumijevano koristi sistem ISO/IEC 8859-1 (odnosno Latin 1), ili tačnije, sistem Windows-1252 (koji je, zapravo, minorna nadgradnja sistema ISO/IEC 8859-1). Ovo se može promijeniti u postavkama editora. Međutim, zanimljivo je da ovaj editor, čim uoči neki znak koji nije podržan sistemom koji se podrazumijeva (u skladu s tekućim postavkama), kao što je neki znak iz bosanskog jezika, prilikom snimanja programa automatski mijenja sistem kodiranja u UTF-8, uz prijavu upozorenja da će biti izvršena promjena sistema kodiranja (tako da će, u slučaju prisustva znakova iz bosanskog jezika u programu, program biti sačuvan koristeći UTF-8 sistem kodiranja). Što se tiče odgovora na drugo i treće pitanje, GCC kompajler (koji podrazumijevano dolazi uz CodeBlocks razvojno okruženje), ima podrazumijevani odgovor UTF-8 na oba pitanja, mada se i to može promijeniti u postavkama kompajlera. Inače, odgovori na drugo odnosno treće pitanje su ono što većina kompajlera naziva *ulazni/izvorni* odnosno *izvršni skup znakova* (engl. *input/source character set* odnosno *execution character set*). Zasada sve djeluje usklađeno. Međutim, na MS Windows operativnim sistemima, odgovor na četvrto pitanje ovisi od konkretnih lokalnih postavki operativnog sistema (recimo, koji se podrazumijevani jezik koristi), ali taj odgovor podrazumijevano nikada nije UTF-8, i to je upravo ono što pravi probleme. Recimo, konzola MS Windows operativnih sistema vrlo često podrazumijevano koristi prastari Windows-437 sistem kodiranja znakova (pogotovo u Sjedinjenim Američkim Državama), naslijeđen još s prastarih IBM računara, ponekad se susreće i neki od ISO/IEC 8859 standarda, ali UTF-8 nikada. Stoga je sasvim očekivano da ispis neće biti u skladu s očekivanjima.

Sad, kada smo vidjeli u čemu je problem, razmotrimo i kako ga riješiti. Ukratko, potrebno je postići da odgovor na sva četiri spomenuta pitanja bude isti. Pitanje je na koji odgovor ići i kako to postići. Generalno, danas je najšire podržan sistem kodiranja UTF-8, pa je logično ići upravo da odgovor na sva četiri pitanja bude upravo UTF-8. Počnimo od četvrtog pitanja. Činjenica je da konzole gotovo svih modernih operativnih sistema bez problema podržavaju UTF-8. Probleme jedino prave Microsoftovi operativni sistemi iz MS Windows porodice operativnih sistema, dijelom zbog "tvrdoglavosti" koja je itekako prisutna u firmi Microsoft, dijelom zbog straha od gubitka kompatibilnosti s nekim starijim programima koji nisu podržavali UTF-8. Sretna okolnost je što je MS Windowsovu konzolu moguće "natjerati" da korektno interpretira znakove kodirane prema UTF-8 sistemu. Za tu svrhu, potrebno je operativnom sistemu poslati odgovarajuću komandu, koja konkretno glasi "`chcp 65001 >nul`" (ovdje je "`chcp`" skraćenica od "change code page", 65001 je Microsoftova interna kodna oznaka za UTF-8 sistem, a "`>nul`" govori da ne želimo prikaz rezultata izvršavanja ove komande na ekranu. Da bismo poslali neku komandu operativnom sistemu iz programskog jezika C++, možemo koristiti funkciju "`system`" iz biblioteke s zaglavljem "`cstdlib`", kojoj se unutar znakova navoda šalje komanda koju želimo poslati operativnom sistemu (ovo je dovoljno uraditi samo jednom, na početku izvršavanja programa). Dakle, ukoliko je operativni sistem na kojem radimo neki od MS Windows operativnih sistema i ukoliko je odgovor na prva tri sporna pitanja UTF-8 (što će, recimo, biti slučaj ukoliko radimo u CodeBlocks razvojnom okruženju), sljedeći program će dati korektan ispis:

```
#include <iostream>
#include <cstdlib>

int main() {
  std::system("chcp 65001 >nul");
  std::cout << "Ćevabdžinica Željo\ns Baščaršije";
  return 0;
}
```

Ovim još uvijek ni izdaleka nisu riješeni svi problemi. Prvi problem je šta ukoliko želimo da imamo univerzalan program koji će raditi na bilo kojem operativnom sistemu, s obzirom da konstrukcija "`chcp 65001 >nul`", koja je ovdje korištena, nema smisla na drugim operativnim sistemima i može dovesti do pojave greške. Drugi problem je šta ukoliko odgovor na prva tri sporna pitanja nije UTF-8. Prvo ćemo riješiti prvi problem. Ideja je da se testira da li je operativni sistem neki od MS Windows operativnih sistema, i da se sporna komanda izvrši samo ako jeste. Mada ne postoji standardni način detekcije koji se operativni sistem koristi, praktično svi kompajleri imaju međusobno dogovorene nestandardne načine da se to postigne. Tako, postoji dogovor da svi kompajleri koji se izvode na nekom

od 32-bitnih ili 64-bitnih MS Windows operativnih sistema (dakle, svim verzijama koje se danas koriste) obavezno definiraju jedan poseban simbol koji se naziva "`_WIN32`". U suprotnom, ovaj simbol neće biti definiran. Da li je ovaj simbol definiran ili ne, može se provjeriti korištenjem direktive "`#ifdef`", na način koji je vidljiv iz sljedećeg primjera:

```
#include <iostream>
#include <cstdlib>

int main() {
  #ifdef _WIN32
    std::system("chcp 65001 >nul");
  #endif
  std::cout << "Ćevabdžinica Željo\ns Baščaršije";
  return 0;
}
```

Na ovaj način, komanda "`chcp 65001 >nul`" će biti poslana samo ukoliko se radi o nekom od MS Windows operativnih sistema, u suprotnom će biti ignorirana. Ovim su riješeni problemi vezani za ispis naših slova, ukoliko je odgovor na prva tri sporna pitanja UTF-8, neovisno od operativnog sistema koji se koristi (s obzirom da konzole praktično svih, operativnih sistema, osim Microsoftovih koje svakako tretiramo posebno, podržavaju UTF-8 bez problema).

Pozabavimo se sada rješavanjem preostalih pitanja. Treće pitanje je vrlo lako riješiti. Istina je da standardi jezika C i C++ ne propisuju ništa o tome koji će se sistem šifriranja koristiti u generiranom prevedenom mašinskom kôdu ukoliko se unutar teksta između navodnika nađe neki od znakova koji se ne nalaze u skupu znakova koji je podržan ASCII sistemom. Odluka o tome se tada prepušta kompajleru i obično ju je moguće podesiti negdje u postavkama kompalera. Kod većine današnjih kompajlera, podrazumijevano se koristi UTF-8, ali postoje izuzeci. Recimo, Microsoft Visual C++ kompajler (ili, skraćeno, MSVC), podrazumijevano koristi šifriranje koje je u skladu s regionalnim postavkama koje su odabrane prilikom njegove instalacije. Tako će, ukoliko je kompajler instaliran da se koristi na ovdašnjim prostorima, vrlo vjerovatno podrazumijevano šifriranje u prevedenom mašinskom kôdu biti prilagođeno ISO/IEC 8859-1 (odnosno Latin 2) standardu, u skladu s općom Microsoftovom politikom (mada je ovo moguće promijeniti negdje u postavkama kompajlera). Međutim, počev od standarda C++11 jezika C++ nadalje, postoji vrlo jednostavan način da se programskim putem forsira da sistem šifriranja svih znakova u generiranom prevedenom mašinskom kôdu bude u skladu s UTF-8 sistemom, neovisno od toga kakve su postavke kompajlera i neovisno od toga kako su kodirani znakovi u izvornom programu. Za tu svrhu, dovoljno je staviti prefiks "`u8`" ispred početnog navodnika iza kojeg slijedi tekst koji se ispisuje. Stoga će program poput sljedećeg garantirati da će odgovor na treće i četvrto pitanje biti UTF-8, čime rješavamo i problem s kompajlerima kod kojih odgovor na treće pitanje nije UTF-8:

```
#include <iostream>
#include <cstdlib>

int main() {
  #ifdef _WIN32
    std::system("chcp 65001 >nul");
  #endif
  std::cout << u8"Ćevabdžinica Željo\ns Baščaršije";
  return 0;
}
```

Ostaje još da razmotrimo prva dva pitanja. Jasno je da je iz samog programa nemoguće utjecati na to koji će sistem šifriranja koristiti editor u kojem se piše program, kao ni na to koji će sistem šifriranja kompajler podrazumijevati da je korišten u izvornom programu koji treba prevesti u mašinski kôd. Stoga, izgleda da nije moguće ni na kakav način uskladiti da odgovor na prva dva pitanja bude UTF-8, osim podešavanjem postavki editora odnosno kompajlera (srećom, to je obično moguće postaviti). Međutim, i ovo je moguće riješiti, po cijenu smanjene čitljivosti, s obzirom da postoji način da se unutar teksta između navodnika naznači da želimo neki znak koji nije podržan ASCII sistemom, ali na način da se pri tome ne upotrijebe nikakvi znakovi koji izlaze izvan ASCII sistema (tako da, sâm program, neće sadržavati nijedan znak koji nije podržan ASCII sistemom). Naime, unutar teksta između navodnika, mogu se ubacivati neke sekvence znakova koje imaju posebna značenja, a koje počinju znakom "`\`" (tzv. *escape sekvence*). Najpoznatija među njima je dobro poznata sekvenca "`\n`", koja dovodi do prelaza u novi red. Isto tako, postoje i sekvence koja omogućavaju da se zada bilo koji znak podržan UNICODE standardom, navodeći umjesto sâmog znaka njegovu kodnu tačku. Te sekvence počinju s "`\u`", nakon

čega slijedi četverocifreni broj u heksadekadnom brojnom sistemu, koji predstavlja željenu kodnu tačku. Recimo, kako veliko slovo "Č" ima kodnu tačku 268, odnosno 10c u heksadekadnom brojnom sistemu, veliko slovo "Č" možemo predstaviti kao sekvencu "`\u010c`" (dodata je jedna vodeća nula da bi se dobio četverocifreni heksadekadni broj). Kodne tačke u heksadekadnom brojnom sistemu za sve specijalne znakove bosanskog jezika prikazane su u sljedećoj tabeli:

Slovo	Kodna tačka (heksadekadno)
Č	10c
č	10d
Ć	106
ć	107
Đ	110
đ	111
Š	160
š	161
Ž	17d
ž	17e

U skladu s tim, sljedeći program trebao bi proizvesti korektan ispis bez obzira na editor koji je korišten za pisanje programa, kompajler koji je korišten za njegovo prevođenje, te operativni sistem na kojem će se prevedeni program izvršavati:

```
#include <iostream>
#include <cstdlib>

int main() {
  #ifdef _WIN32
    std::system("chcp 65001 >nul");
  #endif
  std::cout << u8"\u0106evabd\u017einica \u017deljo\ns Ba\u0161\u010dar\u0161ije";
  return 0;
}
```

Upravo zbog neovisnosti od postavki editora i kompajlera, preporučuje se da se unutar tekstova za navođenje bilo kojeg znaka koji nije podržan ASCII sistemom šifriranja koriste upravo opisane specijalne sekvence. Jedini nedostatak opisanog pristupa je smanjena čitljivost, jer samo gledanjem programa nije lako vidjeti šta tačno piše unutar teksta, bez konsultiranja prethodno navedene tablice.